MyTownMarket: Full-Stack Marketplace Platform for Local Businesses

MyTownMarket – Project Documentation

1. Project Overview

MyTownMarket is a full-stack marketplace platform designed to help local businesses showcase their products and allow users to browse, order, and pay for them digitally.

The platform supports:

- Business listing
- Product catalog
- Shopping cart
- Order management
- QR based UPI payment
- Cash on Delivery (COD)

The system includes an **Admin panel for business owners** and a **User interface for customers**.

---

2. System Architecture

The project follows a **Full Stack Client–Server Architecture**.

```
Frontend (Next.js / React)
        │
        │ REST API
        ▼
Backend (Node.js + Express)
        │
        │
        ▼
Database (MongoDB)
```

Components

Frontend
→ UI rendering
→ API requests
→ Authentication handling

Backend
→ Business logic
→ API routes
→ Authentication validation
→ File upload management

Database
→ Stores users
→ Businesses
→ Products
→ Orders

---

3. Technology Stack

Frontend

- Next.js
- React.js
- Context API (State management)
- CSS
- Fetch API

Backend

- Node.js
- Express.js
- JWT Authentication
- Multer (Image Upload)

Database

- MongoDB
- Mongoose ORM

Tools

- Git
- GitHub
- VS Code
- Postman (API testing)

---

4. Project Folder Structure

Root Folder

```
MyTownMarket
 ├ Backend
 └ mytownmarket-frontend
```

Backend Structure

```
Backend
│
├ models
│   ├ Business.js
│   ├ Product.js
│   └ Order.js
│
├ routes
│   ├ businessRoutes.js
│   ├ productRoutes.js
│   └ orderRoutes.js
│
├ src
│   ├ config
│   │   └ db.js
│   ├ controllers
│   │   └ userController.js
│   ├ middleware
│   │   └ authMiddleware.js
│   ├ models
│   └ routes
│
```

```
├ uploads
│
└ server.js
```

Frontend Structure

```
mytownmarket-frontend

src
│
├ app
│   ├ login
│   ├ signup
│   ├ protected
│   │   ├ dashboard
│   │   ├ cart
│   │   ├ business
│   │   ├ admin
│
├ context
│   ├ AuthContext.js
│   └ BusinessContext.js
│
└ lib
    └ auth.js
```

---

5. Main Functional Modules

User Authentication

Features:

- Signup
- Login
- JWT Token authentication

Files involved:

- `userController.js`
- `authMiddleware.js`
- `AuthContext.js`

Business Management (Admin)

Admin can:

- Add Business
- Upload Business Image
- Upload QR Code
- Edit Business
- View Orders

Files:

```
businessRoutes.js
Business.js
admin/business/page.js
```

Product Management

Admin can:

- Add products
- Edit products
- Delete products
- Upload product images

Files:

```
productRoutes.js
Product.js
admin/products
```

Shopping Cart

Features:

- Add to cart
- Remove items
- Checkout
- Choose payment type

Files:

```
cart/page.js
BusinessContext.js
```

Payment System

Two payment methods supported:

1. UPI QR Payment

User scans QR code displayed on cart page.

Displayed data:

```
QR Image
UPI ID
```

Admin uploads QR image from admin panel.

---

2. Cash On Delivery (COD)

User selects COD during checkout.

Order is marked as:

```
paymentType: COD
```

---

6. Order Management

Admin can view:

- Customer name
- Products ordered
- Order type
- Payment type (UPI or COD)

Files:

```
orderRoutes.js
Order.js
admin/orders/page.js
```

---

7. API Endpoints

User APIs

```
POST /api/users/signup
POST /api/users/login
```

Business APIs

```
POST /api/business
GET /api/business
PUT /api/business/:id
```

Product APIs

```
POST /api/products
GET /api/products
DELETE /api/products/:id
```

Order APIs

```
POST /api/orders
GET /api/orders
```

---

8. Security Measures

- JWT authentication
- Protected admin routes
- .env for secrets
- Node_modules ignored in Git
- Upload folder ignored in Git

---

9. Deployment Readiness

Project is ready for deployment with:

Backend:

- Render
- Railway
- AWS

Frontend:

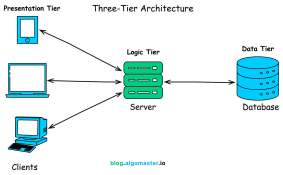
- Vercel
- Netlify

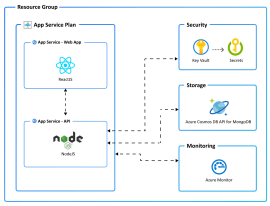
Database:

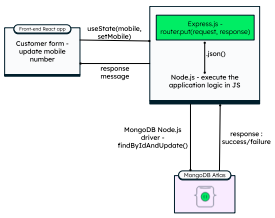
- MongoDB Atlas

1️⃣ System Architecture Diagram

This shows **how your whole system works together**.







4

Architecture Explanation

```
User Browser
      │
      │ HTTP Requests
      ▼
Frontend (Next.js / React)
      │
      │ REST API Calls
      ▼
Backend (Node.js + Express)
      │
      │ Database Queries
      ▼
MongoDB Database
```

Flow in your project

User Flow

```
User → Login / Signup
User → Browse Businesses
User → View Products
User → Add to Cart
User → Checkout
User → Scan QR / COD
Order → Stored in Database
Admin → View Orders
```

Components

Frontend

- Next.js
- React
- Context API
- Fetch API

Backend

- Node.js
- Express
- JWT Authentication
- Multer image upload

Database

- MongoDB
- Mongoose ODM

2️⃣ Database Schema Diagram

This explains **how your MongoDB collections relate**.

4

Collections Used

Your system has **4 main collections**.

User Collection

```
User
-----
_id
name
```

```
email
password
role (admin / customer)
```

Business Collection

```
Business
--------
_id
name
sector
phone
email
address
description
images
qrCode
upiId
```

Product Collection

```
Product
-------
_id
name
description
price
images
business (Business ID)
```

Relationship:

```
1 Business → Many Products
```

Order Collection

```
Order
-----
_id
customerName
phone
items
totalPrice
paymentMethod (UPI / COD)
businessId
createdAt
```

Relationship:

```
User → places → Orders
Business → receives → Orders
Order → contains → Products
```

3️⃣ API Documentation Table

This is **very important for professional projects**.

User APIs

Method	Endpoint	Description
POST	`/api/users/signup`	Register new user

POST	`/api/users/login`	Login user

Business APIs

Method	Endpoint	Description
POST	`/api/business`	Add new business
GET	`/api/business`	Get all businesses
GET	`/api/business/:id`	Get business details
PUT	`/api/business/:id`	Update business

Product APIs

Method	Endpoint	Description
POST	`/api/products`	Add product
GET	`/api/products/by-business/:id`	Get business products
PUT	`/api/products/:id`	Update product
DELETE	`/api/products/:id`	Delete product

Order APIs

Method	Endpoint	Description
POST	`/api/orders`	Create order
GET	`/api/orders`	Get orders
GET	`/api/orders?business=id`	Orders by business

Payment Logic

UPI Payment

```
User selects UPI
↓
QR Code displayed
↓
User scans QR
↓
Order stored in database
```

COD Payment

```
User selects Cash on Delivery
↓
Order stored in database
↓
Admin fulfills order
```

---

10. Future Improvements

Payment Improvements

- Razorpay integration
- Automatic payment verification

Order Tracking

Add order status:

```
Pending
Accepted
Preparing
Delivered
```

Customer Contact Details

Add fields in orders:

```
Customer Phone
Delivery Address
```

Product Features

- Product search
- Product filters
- Product categories

UI Improvements

- Better dashboard design
- Loading animations
- Mobile responsiveness

Business Analytics

Add dashboard insights:

- Total sales
- Total orders
- Best selling products

---

11. Known Limitations

- QR payment is manual verification
- No live payment gateway
- No push notifications
- No delivery tracking

---

12. GitHub Repository

Project Repository:

```
https://github.com/ShraddhaM1/MyTownMarket
```

---

13. Author

Authors

Shraddha Mhaske
Kejal Kasliwal

Computer Engineering Students